

Physics-Informed Geometry-Aware Neural Operator

Weiheng Zhong, Hadi Meidani *

Department of Civil and Environmental Engineering, University of Illinois at Urbana-Champaign, Champaign, IL, United States of America

ARTICLE INFO

Keywords:

Physics-informed deep learning
Neural operator
Geometry generalization

ABSTRACT

Engineering design problems often involve solving parametric Partial Differential Equations (PDEs) under variable PDE parameters and domain geometry. Recently, neural operators have shown promise in learning PDE operators and quickly predicting the PDE solutions. However, training these neural operators typically requires large datasets, the acquisition of which can be prohibitively expensive. To overcome this, physics-informed training offers an alternative way of building neural operators, eliminating the high computational costs associated with Finite Element generation of training data. Nevertheless, current physics-informed neural operators struggle with limitations, either in handling varying domain geometries or varying PDE parameters. In this research, we introduce a novel method, the Physics-Informed Geometry-Aware Neural Operator (PI-GANO), designed to simultaneously generalize across both PDE parameters and domain geometries. We adopt a geometry encoder to capture the domain geometry features, and design a novel pipeline to integrate this component within the existing Deep Compositional Operator Network architecture. Numerical results demonstrate the accuracy and efficiency of the proposed method. All the codes and data related to this work are available on GitHub: <https://github.com/uq-group/codes/tree/main/PI-GANO>.

1. Introduction

The finite element method (FEM) as the predominant high fidelity numerical approach for solving partial differential equations (PDEs) involves discretizing a continuous function space using a discrete mesh and solving a high-dimensional linear system [1]. The cost associated with this solution can be prohibitively large particularly in tasks requiring repetitive solutions. An example of such tasks is engineering design which necessitates solving parameterized PDEs over a wide range of PDE parameters and domain geometries for design evaluation [2].

Recently, machine learning techniques have been introduced to accelerate the process of solving PDEs by learning a neural operator as a mapping from variable PDE parameters and/or domain geometry to the PDE solution [3]. Once a neural operator model is successfully trained on a dataset, it can generalize to new, unseen parameters and domain geometries. This is done by a single forward pass through the trained neural network, with minimal computational cost.

Research on neural operators initially was focused on developing various neural operator architectures for operator learning of parametric PDEs in a data-driven way [3–6]. As the first neural operator architecture, Deep Operator Networks (DeepONet) [3] utilize the universal approximation theorem for operators, introducing an architecture that approximates nonlinear operators through learning a collection of basis functions and coefficients. The Fourier Neural Operator (FNO) [4] leverages the Fourier transform to model mappings in the spectral domain, capturing global dependencies and showing superior performance in square domain shapes. Geo-FNO [7] extends FNO to irregular meshes by learning a mapping from an irregular mesh to a uniform mesh.

* Corresponding author.

E-mail addresses: weiheng4@illinois.edu (W. Zhong), meidani@illinois.edu (H. Meidani).

<https://doi.org/10.1016/j.cma.2024.117540>

Received 16 August 2024; Received in revised form 11 October 2024; Accepted 6 November 2024

Available online 26 November 2024

0045-7825/© 2024 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

Table 1

Differences of physics-informed geometry-aware neural operator versus other existing models.

	Data-free	Generalize across PDE parameter representations	Generalize across geometries
DeepONet [3]	–	–	–
FNO [4], WNO [8], LRNO [5] geo-FNO [7]	–	✓	–
GNO [6], GNOT [12], GA-FNO [10], Diff-NO [11]	–	✓	✓
PI-DeepONet [17]	✓	–	–
PI-FNO [20], PI-WNO [21], PI-DCON [22]	✓	✓	–
PI-PointNet [23]	✓	–	✓
PI-GANO	✓	✓	✓

Inspired by FNO, Wavelet Neural Operator (WNO) [8] is also proposed by using wavelet transform instead of Fourier transform to better handle signals with discontinuity and spikes in an irregular domain geometry. While each of these methods has demonstrated potential in specific applications, a common limitation is that each trained model is restricted to a specific domain geometry.

To overcome the limitations of these “geometry-specific” methods, a number of “geometry aware” methods have been developed. The Graph Neural Operator (GNO) [6] employs graph neural networks (GNNs) for operator learning by treating inputs and outputs as graphs, enabling generalization across different domain geometries. Similarly, PointNet [9] facilitates PDE operator learning across diverse geometries by using a set of collocation points to represent the domain geometry. As an extension of FNO, geometry-aware FNO [10] was proposed for PDE solution prediction using a signed distance function (SDF) and point-cloud representations of the geometry. Diffeomorphism Neural Operator [11] combines harmonic mapping approach and the FNO to offer geometry generalization ability to FNO. Additionally, the General Neural Operator Transformer (GNOT) [12] encodes the PDE parameters and domain geometries into tokens and adopt the architecture of Transformer [13] for PDE solution predictions. Using the same idea of GNOT, other transformer-based methods [14–16] were also proposed to address PDEs in varying geometrical settings.

Even though these methods offer the ability of generalizing to various PDE parameters and geometries, they all are data-intensive [17]. To bypass the need for training datasets, which are typically expensive to obtain, physics-informed neural operators, which are inspired by physics-informed neural networks (PINNs) [18], have emerged as an effective approach. This approach integrates the governing PDEs directly into the training process, making it possible to develop a neural operator without any training data obtained from high fidelity simulations, such as FEM. Among these methods, Physics-informed DeepONet (PI-DeepONet) [17] extends the original DeepONet framework by embedding physical laws directly into the loss function with automatic differentiation [19] during training, which can only be applied on a specific mesh. Physics-informed Fourier Neural Operator (PI-FNO) [20] builds upon the original FNO framework, offering generalization across different PDE parameter representations in a square domain while also incorporating physics-informed training methods. Similarly, physics-informed Wavelet Neural Operator (PI-WNO) [21] is also proposed based on the architecture of WNO, implementing physics-informed training for WNO [8] using stochastic projection. Physics-informed Deep Compositional Operator Network (PI-DCON) [22] adopts a pooling layer to capture the global features of the variable PDE parameters, which enables the learning of the PDE operator on any given irregular domain geometry. However, all these physics-informed methods struggle with generalization to varying geometries. A significant advancement to address this limitation is the Physics-informed PointNet (PI-PointNet) [23], which integrates a physics-informed training algorithm into PointNet. Nonetheless, PI-PointNet lacks the capability to generalize across PDE parameters, which limits its application in tackling complex problems in engineering design which also involves varying boundary conditions, etc.

In this study, we tackle the aforementioned challenges by introducing an innovative framework, called Physics-informed Geometry-aware Neural Operator (PI-GANO). This model is inspired by PI-DCON and PI-PointNet and is capable of generalizing across different PDE parameter representations and domain geometry representations simultaneously, including those in irregular domain shapes. The differences between our proposed model versus other existing works are summarized in Table 1. To the best of our knowledge, our proposed model is the first attempt to develop a neural operator that can simultaneously handle varying PDE parameters and domain geometries without need for any training data.

The remainder of this paper is organized as follows. In Section 2, we briefly introduce the problem settings and technical backgrounds for PI-DeepONet and PI-PointNet. Section 3 introduces our model architecture and explains the key points about it. Finally, numerical experiments and detailed performance evaluation of the proposed methods and conclusions are included in Sections 4 and 5.

2. Technical background

2.1. Problem setting

Our goal is to develop an efficient machine learning-based solver for parametric PDEs which are formulated by:

$$\begin{aligned}\mathcal{N}_{\mathbf{x}}[u(\mathbf{x}), k(\mathbf{x})] &= 0, & \mathbf{x} \in \Omega, \\ B_{\mathbf{x}}[u(\mathbf{x})] &= g(\mathbf{x}), & \mathbf{x} \in \partial\Omega,\end{aligned}\tag{1}$$

where Ω is a physical domain in R^d , $\mathbf{x}$ is a d -dimensional vector of spatial coordinates, $\mathcal{N}_{\mathbf{x}}$ is a general differential operator, and $B_{\mathbf{x}}$ is a boundary conditioned operator acting on the domain boundary $\partial\Omega$. Also, $k(\mathbf{x})$ refers to the parameters of the PDE, which can

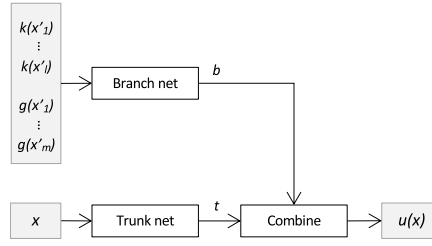


Fig. 1. The architecture of the DeepONet is shown. The input to the branch net is the sampled discrete representations of variable parameters $k(\mathbf{x})$ and $g(\mathbf{x})$, with l and m discretization points, respectively.

include the coefficients and forcing terms in the governing equation, $g(\mathbf{x})$ denotes the boundary conditions, and $u(\mathbf{x})$ is the solution of this PDE for a given set of parameters and boundary conditions.

We seek to solve this PDE across different parameters and geometries. Let $\Omega = \{\Omega_1, \Omega_2, \dots, \Omega_n\}$ denote the set of variable geometries. For each geometry Ω_i , we have a corresponding set of PDE parameters, i.e. coefficients denoted by $k_i(\mathbf{x})$ and boundary conditions denoted by $g_i(\mathbf{x})$. Therefore, we employ a neural network model to approximate the operator $\mathcal{M}$ defined by:

$$\mathcal{M} : \{k_i(\mathbf{x}), g_i(\mathbf{x}), \Omega_i\} \rightarrow u_i(\mathbf{x}), \quad \mathbf{x} \in \Omega_i, \quad \Omega_i \in \Omega, \quad (2)$$

where the inputs to the operator include information on both PDE parameters and domain geometries. Existing physics-informed neural operators primarily focus on learning the PDE operator with variations in either the PDE parameters or geometry. Our proposed framework handles both of these variation types. This is done by using two model ingredients: (1) one based on PI-DeepONet to handle varying PDE parameters and (2) one inspired by PI-PointNet to encode varying domain geometries. In the next two sections, we present the background for PI-DeepONet and PI-PointNet.

2.2. Parameter encoding with physics-informed DeepONet

DeepONet was proposed to solve parametric PDEs by approximating a domain-specific operator $\mathcal{M}$ given by

$$\mathcal{M} : \{k_i(\mathbf{x}), g_i(\mathbf{x})\} \rightarrow u_i(\mathbf{x}), \quad \mathbf{x} \in \Omega \quad (3)$$

where Ω is a given domain geometry. Specifically, DeepONet approximates $\mathcal{M}$ by a neural network model $\mathcal{U}_\theta(\mathbf{x}, k(\mathbf{x}'), g(\mathbf{x}''))$, where $\mathbf{x}$ denotes the coordinates at which the solution u is to be calculated, and $\mathbf{x}'$ and $\mathbf{x}''$ denote the coordinates at which the values of coefficients and boundary conditions, respectively, are available.

As shown in Fig. 1, the architecture of DeepONet is composed of two separate neural networks referred to as the “branch net” and “trunk net”, respectively. Both the branch net and trunk net are simply multilayer perceptrons (MLP). The DeepONet prediction is given by

$$\mathcal{U}_\theta(\mathbf{x}, k(\mathbf{x}'), g(\mathbf{x}'')) = \sum_{i=1}^q b_i t_i. \quad (4)$$

where the q -dimensional features embedding $\mathbf{b} = [b_1, b_2, \dots, b_q]$ is the output of the branch net, and the q -dimensional coordinate embedding $\mathbf{t} = [t_1, t_2, \dots, t_q]$ is the output of the trunk neural networks. To calculate $\mathbf{t}$, the trunk net takes the coordinates of a collocation point, $\mathbf{x}$, as input. To calculate $\mathbf{b}$, the branch net takes $[k(\mathbf{x}'_1), k(\mathbf{x}'_2), \dots, k(\mathbf{x}'_l), g(\mathbf{x}''_1), g(\mathbf{x}''_2), \dots, g(\mathbf{x}''_m)]$ as input. This input is the parameter function $k(\cdot)$ evaluated at a collection of sampled locations $\{\mathbf{x}'_j\}_{j=1}^l$ and the boundary function $g(\cdot)$ evaluated at a generally different collection of sampled locations $\{\mathbf{x}''_j\}_{j=1}^m$.

In order to set up the training loss for the model training, we first obtain N realizations of branch net input, where each realization is parameter function $k(\cdot)$ obtained at locations $\mathbf{x}'$ and BC function $g(\cdot)$ obtained at locations $\mathbf{x}''$. Then the physics-informed training loss of the neural operator $\mathcal{L}_{\text{PINO}}$ is given by:

$$\mathcal{L}_{\text{PINO}} = \mathcal{L}_{\text{PDE}} + \alpha \mathcal{L}_{\text{BC}} \quad (5)$$

where

$$\begin{aligned} \mathcal{L}_{\text{PDE}} &= \frac{1}{N} \sum_{i=1}^N (\mathcal{N}_x[\mathcal{U}_\theta(\mathbf{x}, k_i(\mathbf{x}'), g_i(\mathbf{x}'')), k_i(\mathbf{x}')])^2, & \mathbf{x} \in \Omega, \\ \mathcal{L}_{\text{BC}} &= \frac{1}{N} \sum_{i=1}^N (\mathcal{B}_x[\mathcal{U}_\theta(\mathbf{x}, k_i(\mathbf{x}'), g_i(\mathbf{x}''))] - g_i(\mathbf{x}''))^2, & \mathbf{x} \in \partial\Omega, \end{aligned} \quad (6)$$

where α is the trade-off coefficient between the PDE residual loss term and the initial and boundary condition loss term. The optimal neural network parameters θ are found by minimizing the total training loss $\mathcal{L}_{\text{PINO}}$ with exact derivatives computed using automatic differentiation [19]. After the model training, for any new realization of the parameters, the well-trained DeepONet can predict the corresponding solution directly in a specific domain geometry Ω . To ensure the efficiency of model training, we implement stochastic gradient descent algorithm to update model parameters by random sampling of collocation points in each epoch.

2.3. Geometry encoding with physics-informed PointNet

PI-PointNet focus on approximating the PDE operator across a set of domain geometries. In its original form, it does not handle varying PDE parameters. Specifically, let the operator $\mathcal{M}$ be given by

$$\mathcal{M} : \{\Omega_i\} \rightarrow u_i(\mathbf{x}), \quad \mathbf{x} \in \Omega_i, \quad \Omega_i \in \Omega. \quad (7)$$

Each domain geometry Ω_i is represented by a set of s_i collocation points inside the domain, denoted by $\{\mathbf{x}_j\}_{j=1}^{s_i}$. PI-PointNet takes these coordinates as input, and predicts the PDE solution on the input collocation points $\{u(\mathbf{x}_i)\}_{i=1}^{s_i}$. The model parameters are estimated in a physics-informed training using the same loss function shown in Eq. (5).

The architecture of the PointNet consists of two multilayer perceptrons $\mathcal{U}_1(\cdot, \theta_1)$ and $\mathcal{U}_2(\cdot, \theta_2)$, where θ_1 and θ_2 are trainable parameters. For each geometry input Ω_i , it first generates the hidden embeddings $\mathbf{h}$ of the input collocation points by:

$$\{\mathbf{h}_j\}_{j=1}^{s_i} = \{\mathcal{U}_1(\mathbf{x}_j, \theta_1)\}_{j=1}^{s_i}. \quad (8)$$

Then a Maxpooling layer [24] is applied to hidden embeddings to obtain a global feature $\mathbf{G}^i$ of the i th geometry:

$$\mathbf{G}^i = \text{Maxpool}(\{\mathbf{h}_j\}_{j=1}^{s_i}), \quad (9)$$

where Maxpool refers to the max-pooling operation which calculates the maximum value in each embedding dimension over all the s_i points. The global feature is then concatenated to each of the hidden embeddings, providing a unique embedding $\mathbf{H}_j^i$ of coordinate $\mathbf{x}_j$ in the i th geometry by

$$\mathbf{H}_j^i = [\mathbf{h}_j \parallel \mathbf{G}^i]. \quad (10)$$

The final PDE solution over the whole domain $\{u(\mathbf{x}_i)\}_{i=1}^{s_i}$ is computed by

$$\{u_i(\mathbf{x}_j)\}_{j=1}^{s_i} = \{\mathcal{U}_2(\mathbf{H}_j^i, \theta_2)\}_{j=1}^{s_i}. \quad (11)$$

3. Methodology

In this section, we introduce our proposed Physics-informed Geometry-aware Neural Operator (PI-GANO) for PDE operator learning. Our approach is primarily based on the PI-DCON architecture and draws inspiration from PI-PointNet. Our main goal in the model architecture design is to enable discretization-independent generalizability for both variable parameters and variable geometry. To do so, we seek to integrate the parameter generalizability of PI-DCON with the geometry generalizability of the PI-PointNet architecture, in a way that facilitates physics-informed training.

In the following sections, we discuss how our architecture is inspired from PI-PointNet and present the mathematical representations of the geometry-coordinate embedding. Then, we discuss the architecture of DCON and its limitation in handling varying geometries in Section 3.2, followed by a detailed discussion of our proposed model architecture in Section 3.3.

3.1. Geometry-coordinate encoding

Inspired by PI-PointNet, our approach aims to integrate the coordinate of collocation points with information on the domain geometry to compute the PDE solution. This enables the model to generalize across different domain geometries. We first present the mathematical representation for the geometry-coordinate embedding that encapsulate both local coordinate features and global geometric features.

We first propose a multi-layer perception layers (MLP) $\mathcal{U}_G(\cdot, \theta_G)$ to capture the feature of the domain geometry. Specifically, we represent a given domain geometry, denoted by Ω_i , using a number of collocation points, with coordinates $\{\mathbf{x}_j\}_{j=1}^{s_i}$. We compute the high-dimensional features of each collocation point using the MLP. Then, similarly to PI-PointNet, we use a pooling layer to extract the global feature of the geometry in each embedding dimension. Hence, the global features of the i th domain geometry $\mathbf{G}^i$ is computed by:

$$\mathbf{G}^i = \text{AvgPool}(\{\mathbf{h}_j^i\}_{j=1}^{s_i}), \quad (12)$$

where $\{\mathbf{h}_j^i\}_{j=1}^{s_i} = \{\mathcal{U}_G(\mathbf{x}_j^i, \theta_G)\}_{j=1}^{s_i}$, and θ_G are trainable parameters of the MLP. We consider three options for selecting the collocation points: (1) selecting points only on the boundary segments that vary, (2) selecting points on the entire boundary, and (3) selecting points inside the entire domain. As discussed in Section 4.4, selecting points only on the variable boundary segments led to the best performance. Additionally, we select the Average-pooling layer instead of other pooling layers as it slightly enhanced the model performance in our experiments. The detailed numerical comparison of the effects of different pooling layers are presented in Section 4.4.

Next, we compute the local coordinate feature of the collocation point on which the PDE solution is to be calculated. Specifically, for $\mathbf{x}_j^i$ in domain geometry Ω_i , this is done via a linear layer parameterized by W_i and B_i and an activation function σ :

$$\mathbf{h}_j^i = \sigma(W_i \mathbf{x}_j^i + B_i). \quad (13)$$

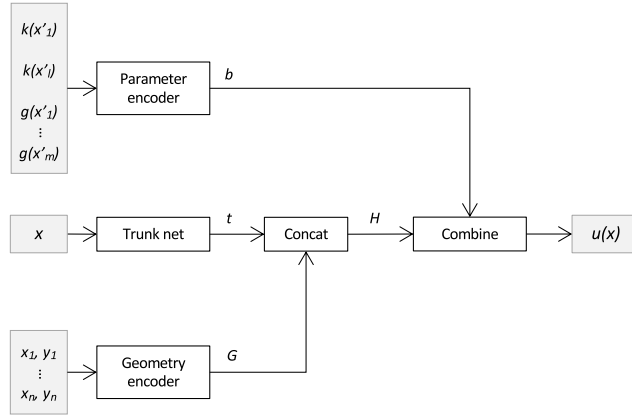


Fig. 2. A schematic of how information on the domain geometry can be integrated with the location and parameter embeddings. The variable inputs include l and m discrete values of parameter functions, together with the coordinates of n points on the domain boundary.

Then we concatenate the geometry global feature G^i with each local coordinate feature h_j^i to obtain the geometry-coordinate embedding H_j^i of each collocation point x_j^i in the geometry Ω_i by:

$$H_j^i = [h_j^i \parallel G^i]. \quad (14)$$

Using these geometry-coordinate embeddings H and the PDE parameter features, we can employ various novel model architectures to predict the PDE solutions. A schematic of our proposed model is shown in Fig. 2. In this work we focus on the case where PDE parameters are not required to be represented using the same number of discretization points, which is especially limiting for variable geometries. Therefore, the numerical experiments are conducted using the PI-DCON architecture, as it is, to the best of the authors' knowledge, the only physics-informed model capable of generalizing to different function discretizations.

3.2. Deep compositional operator network

Our model is built upon on the architecture of Deep Compositional Operator Network [22]. Without loss of generality, let us consider a simplistic case where the variability is only in boundary conditions, denoted by $g(x)$. Our goal is then to find an approximation for the following operator,

$$\mathcal{M} : \{g(x), \Omega\} \rightarrow u(x), \quad x \in \Omega. \quad (15)$$

We can approximate this PDE operator with the architecture of Deep Compositional Operator Network (DCON). Using DCON, the PDE parameters are represented in a discrete form by evaluating function g on a finite number of sampled coordinates $[x'_1, x'_2, \dots, x'_m]$, i.e. $g = [g(x'_1), g(x'_2), \dots, g(x'_m)]$. A multi-layer perceptron, $U_b(\cdot, \theta_b)$, is applied to it with a Max-pooling layer to derive the low-dimensional feature embedding b by:

$$b = \text{Maxpool}(\{U_b(g(x'_j), \theta_b)\}_{j=1}^m), \quad (16)$$

where θ_b are trainable parameters. After computing the parameter embedding b , the PDE operator $\mathcal{M}$ is approximated by a compositional version of DeepONet [3] with multiple stacked operator layers $\{\mathcal{O}_j\}$

$$\mathcal{M} \approx \mathcal{M}_\theta = \text{sum}\{b \odot \dots \odot \mathcal{O}_2(b \odot \mathcal{O}_1(b \odot t(x)))\}, \quad (17)$$

where θ denotes all the trainable parameters of the operator layers, described later, $\odot$ refers to component-wise multiplication, $t : \mathbb{R}^d \rightarrow \mathbb{R}^q$ is a learnable nonlinear mapping from coordinates to hidden embedding, $\mathcal{O} : \mathbb{R}^q \rightarrow \mathbb{R}^q$ is a learnable nonlinear mapping of the operator layers, and $\text{sum}(\cdot)$ returns the summation of the components of a vector. In this work, we use a single linear layer with an activation function as the mapping $\mathcal{O}$, leading to the following operator approximation

$$\mathcal{M} \approx \mathcal{M}_\theta = \text{sum}\{b \odot \dots (b \odot (W_O^2 \sigma(b \odot (W_O^1 \sigma(W_t x + B_t) + B_O^1)) + B_O^2))\}, \quad (18)$$

where $\sigma(\cdot)$ represents the nonlinear activation function, $W_t \in \mathbb{R}^{q \times d}$, $B_t \in \mathbb{R}^q$ are trainable parameters (weights and biases) of the mapping $t(\cdot)$, and $W_O^j \in \mathbb{R}^{q \times q}$, $B_O^j \in \mathbb{R}^q$ are trainable parameters of the j th operator layer, $\mathcal{O}_j$.

It should be noted that this operator is expected to have limitation in handling variation in domain geometry. That is, for a problem with the same parameter function $g(x)$ but two different domains Ω_1 and Ω_2 , the predictions at a given coordinate x , that falls within both domains, will be identical, i.e.

$$\mathcal{M}_\theta(g(x), \Omega_1) = \mathcal{M}_\theta(g(x), \Omega_2). \quad (19)$$

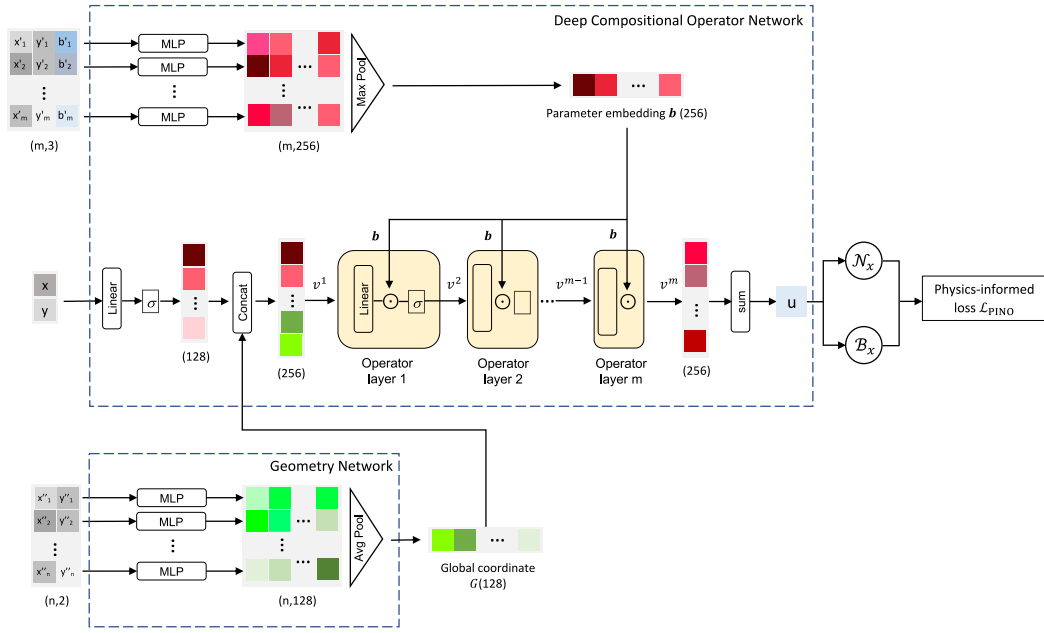


Fig. 3. The architecture of the physics-informed Geometry aware Neural Operator is shown. The gray blocks represent the coordinates and the blue blocks represent the function values. The red blocks represent the hidden embeddings. The value of b_i is the boundary condition value evaluated on (x'_i, y'_i) . The value of u is the solution value evaluated on (x, y) . It should be noted that no activation function is included in the last operator layer. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

This shows that while DCON can still predict solution for any collocation points in a varied or unseen domain, it fails to produce unique predictions for distinct domains. This suggests that the model may exhibit poor performance especially when changes in domain geometry are substantial.

3.3. Proposed model architecture

We seek to develop a model that makes a different prediction at the same spatial coordinate when the domain geometry is modified. To encode the domain geometry into the existing architecture of DCON, we introduce PI-GANO, which combines the representations for geometry-coordinate embedding and DCON.

In our proposed model architecture, the coordinate-geometry embedding H_j^i will be input to a series of operator layers to produce the PDE solution. Finally, the PDE prediction of our model is given by:

$$u_i(x_j^i) \approx \mathcal{M}_\theta(x_j^i, g_i(x), \Omega_i) = \text{sum}\{b \odot \dots (b \odot (W_O^2 \sigma(b \odot (W_O^1 [\sigma(W_i x_j^i + B_i) \parallel G^i] + B_O^1)) + B_O^2))\}, \quad (20)$$

where G^i is the geometry embedding of i th geometry Ω_i computed by Eq. (12). The details about the proposed model architecture are shown in Fig. 3.

Although our model utilizes a similar architecture to PI-PointNet for generating geometry embeddings and coupling global and local features, there are fundamental differences between our architecture and that of PI-PointNet. In the physics-informed training algorithm, we estimate the network parameters by minimizing the PDE residual. This residual in general involves spatial and temporal derivatives which means taking the derivative with respect to the input layer. Therefore, the backward propagation path for derivative calculation using auto differentiation plays a significant role in the quality of physics-informed training.

Fig. 4 shows the computation graphs of PI-PointNet (top) and our model (bottom) for PDE solution in a two-dimensional domain. Solid arrows indicate the forward computation steps, and dashed arrows show the backward propagation steps for derivative calculation. If the two MLPs in our model are identical, it appears that our model and PI-PointNet would share the exact same computation path for PDE solution prediction. However, in our model, the computation of local (collocation point-based) features is independent from that of global features. This results in different computation paths for the derivatives of the model. For instance, in PI-PointNet, the first order derivative of u_j with respect to coordinate x_j is formulated by:

$$\frac{\partial u_j}{\partial x_j} = \frac{\partial u_j}{\partial H_j} \cdot \left[\frac{\partial H_j}{\partial h_j} \parallel \frac{\partial H_j}{\partial G} \frac{\partial G}{\partial h_j} \right] \cdot \frac{\partial h_j}{\partial x_j}, \quad (21)$$

where $\parallel$ represents the operation of concatenation. However, the derivative approximation in our model architecture is simpler than Eq. (21), which is formulated by:

$$\frac{\partial u_j}{\partial x_j} = \frac{\partial u_j}{\partial H_j} \cdot \frac{\partial H_j}{\partial h_j} \cdot \frac{\partial h_j}{\partial x_j}. \quad (22)$$

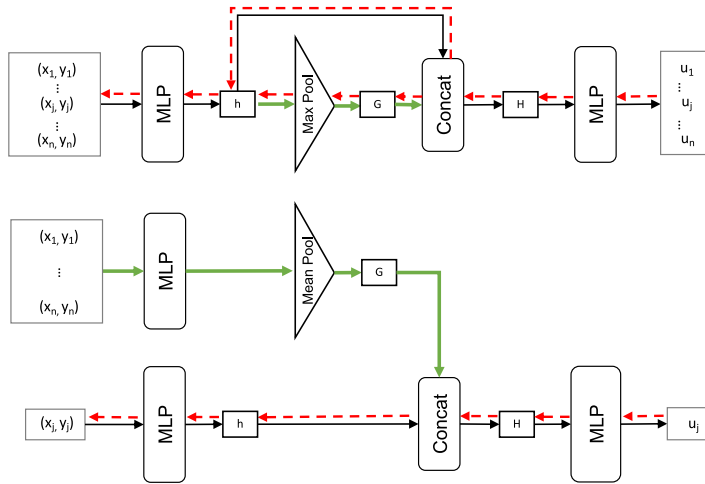


Fig. 4. The differences between our model architecture and PI-PointNet are shown. The black solid arrows are forward computation path for PDE solution prediction, while the red dashed arrows are backward computation path for derivative calculation of the approximated function. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Comparing Eqs. (21) to (22), it is evident that the differentiation process in our proposed architecture involves calculating only the derivatives of MLPs, whereas the PI-PointNet's differentiation process also involves taking the derivative of the Max-pooling layer. Based on the Universal Approximation Theorem for neural networks [25], an MLP can approximate any continuous function and its derivatives within certain error bounds. This is while automatic differentiation over the Max-pooling layer, negatively impacts the model's performance. The numerical results presented in Section 4.3 will further show the differences in performance between these two back-propagation paths.

4. Numerical results

In this section, we numerically evaluate the accuracy of the proposed model for solving parametric differential equations. We compare our models with Physics-informed DCON (PI-DCON), Physics-informed PointNet (PI-PointNet). In order to add PDE parameter generalizability to PI-PointNet, we will also introduce a modified version of PI-PointNet, named PI-PointNet* (as explained in Section 4.1). This modified model, as an additional baseline model, offers a more fair comparison of effectiveness in generalization to variations in both PDE parameters and domain geometry.

In the numerical experiments, we use the following default settings unless mentioned otherwise. We use the hyperbolic tangent function (Tanh) [26] as our activation function to ensure the smoothness in high-order derivatives of the models. For PI-DCON and our model architecture, we use three operator layers of width 512. For the architecture of PI-PointNet and PI-PointNet*, we use three hidden layers of width 512. Adam is the default optimizer with the following default hyper-parameters: $\beta_1 = 0.9$ and $\beta_2 = 0.999$ [27]. Each model is trained on 70% of the sampled PDE parameters and validated on 10% of the sampled PDE parameters. Well-trained models predict the PDE solution for the remaining 20% of the sampled PDE parameters. There are two hyper-parameters used in model training: learning rate and coordinate sampling size ratio in each epoch for computing the PDE residual. We implemented grid search [28] to obtain the best set of hyper-parameters and report the corresponding model performance. The possible values of learning rate are 0.001, 0.0005, 0.0002, and 0.0001, and the possible values of coordinate sampling size ratio are 0.3, 0.2, 0.1, 0.05. For the remaining hyperparameters related to the architecture of DCON, we selected their values based on the hyperparameter studies presented in [22]. Model training is performed on an NVIDIA P100 GPU using a batch size of 20. We collect the parameters of PDEs by Monte Carlo Simulation [29] of stochastic processes [30] and derive the solution of PDEs based on the Finite Element Method using Matlab [31]. For each of the two problems investigated in our work, we generated 500 PDE samples for the experiments. In contrast to the work in [22], which focuses on mesh generalization, we used a consistent resolution across all FEM results in our study, as our primary focus in this study is on generalization across different domain geometries.

4.1. A modified PI-PointNet as a baseline

In its original form, PI-PointNet is not designed for generalization across different PDE parameters. Therefore, to create a baseline with this generalization capability, we have modified it to include a part which encodes PDE parameter information into the model inputs. In this modified architecture, called PI-PointNet*, if the function values of PDE parameters are available, they are concatenated with the coordinates of the collocation points as the model input. If not available, zeros are concatenated instead. This setup ensures that both the PDE parameters and the domain geometry are provided to the model, allowing the PDE solution to depend on both factors. Fig. 5 shows the architecture of PI-PointNet* for the scenario where the only variable PDE parameter is the boundary conditions, where n_1 is the number of collocation points on the boundary and n_2 is the number of collocation points inside the domain.

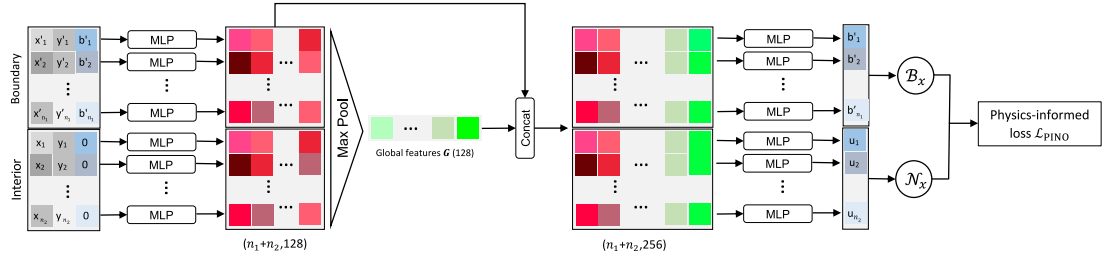


Fig. 5. The architecture of the modified physics-informed PointNet (PI-PointNet*).

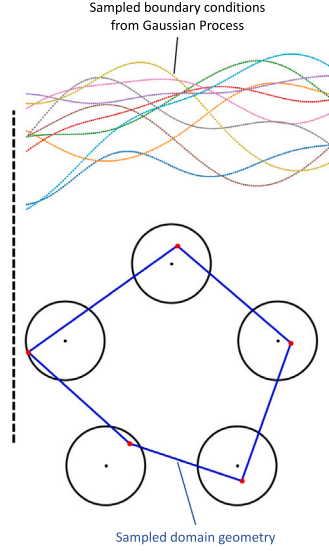


Fig. 6. The boundary conditions in our Darcy flow experiment setting shown on a random geometry. The boundary conditions $g(x)$ are samples from a Gaussian process.

4.2. Experiment setups

4.2.1. A Darcy flow problem

As the first example, we consider a Darcy flow problem over variable domain geometries and boundary conditions. A motivation for creating a fast geometry-aware solver is to facilitate the engineering design problem that seeks the optimal shape of a filtration system that enhances the fluid flow through the porous media. Let us specifically consider a two-dimensional Darcy flow problem in a five-edge polygon, where the steady state solution is described by,

$$\begin{aligned} -\nabla(k(x, y)\nabla p(x, y)) &= f(x, y), \quad (x, y) \in \Omega, \\ p(x, y) &= g(x, y), \quad (x, y) \in \partial\Omega. \end{aligned} \quad (23)$$

where $p(x, y)$ is the pressure, $k(x, y)$ is the permeability field, $f(x, y)$ is the source term, and $g(x, y)$ is the prescribed pressure on the domain boundaries $\partial\Omega$. Following the same setting as [32], we consider $k(x) = 1$ and $f(x) = 10$, where $x = (x, y)$ denotes the coordinates. To create variable domain geometries, we consider an analytical expressions, where the coordinates of the five vertices of the polygon is varied. To do so, we first draw five random coordinates to be the center points for the five circles of radius of 0.25 as shown in 6. These five center points are uniformly drawn on the perimeter of the circle, with the radius of 1 centered at $(0, 0)$. Inside each of the five circles, we will randomly generate one point to serve as a vertex of the polygon-shaped domain.

The boundary $\partial\Omega$ in this example is chosen to be the edges of the polygon, on which we impose the nonzero pressure function $g(x, y)$. The imposed pressure is assumed to follow a zero mean Gaussian process with a covariance kernel that is only dependent on the horizontal component of the distance, i.e.,

$$\begin{aligned} g(x, y) &\sim \mathcal{GP}(0, K(x, x')), \\ K(x, x') &= \exp\left[-\frac{(x - x')^2}{2l^2}\right], \quad l = 1. \end{aligned} \quad (24)$$

We employ the Gaussian process model of Eq. (24) to generate $N = 500$ different imposed pressure functions. Then, given each of these N sampled functions, $\{g_i\}_{i=1}^N$, we solve the PDE using the Finite Element Method (FEM) [31]. Specifically, for the i th realized domain Ω_i and BC function g_i , let M_i be the number of discretization locations on $\partial\Omega_i$, and $\mathbf{X}_{\text{BC},i}$ be the coordinates of these locations, i.e.

$$\mathbf{X}_{\text{BC},i} = \{(x_j^i, y_j^i)\}_{j=1}^{M_i}, \quad (25)$$

where M_i is the size of the discretization generated for the i th sampled geometry. We then collect all the boundary conditions information for the i th geometry in $\mathbf{G}_i$, i.e.

$$\mathbf{G}_i = \{\mathbf{X}_{\text{BC},i}, g_i(\mathbf{X}_{\text{BC},i})\}, \quad (26)$$

where $g_i(\mathbf{X}_{\text{BC},i})$ is the i th sampled BC function evaluated at the coordinates $\mathbf{X}_{\text{BC},i}$.

Given this setup, we seek to learn the nonlinear mapping $\mathcal{M}_{\text{Darcy}}$ that transforms a given boundary function $g(x, y)$ on $\partial\Omega$ to the pressure field $p(x, y)$ in the entire domain, i.e.,

$$\mathcal{M}_{\text{Darcy}} : g(x, y) \rightarrow p(x, y). \quad (27)$$

Specifically, for a 2D problem, the neural operator $p_\theta(x, y, \mathbf{G})$ is trained using the loss function

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \alpha \mathcal{L}_{\text{BC}}(\theta), \quad (28)$$

where α is the weighing hyperparameters, set to be $\alpha = 500$ to ensure similar orders of magnitude; $\mathcal{L}_{\text{PDE}}(\theta)$ and $\mathcal{L}_{\text{BC}}(\theta)$ are the PDE residual and the BC loss function, given by

$$\begin{aligned} \mathcal{L}_{\text{PDE}}(\theta) &= \frac{1}{N} \sum_{i=1}^N \left[\frac{\partial^2 p_\theta(x, y, \mathbf{G}_i)}{\partial x^2} + \frac{\partial^2 p_\theta(x, y, \mathbf{G}_i)}{\partial y^2} + 10 \right]^2, \quad (x, y) \in \Omega \\ \mathcal{L}_{\text{BC}}(\theta) &= \frac{1}{N} \sum_{i=1}^N \left\{ \frac{1}{M_i} \sum_{j=1}^{M_i} [p_\theta(x_j^i, y_j^i, \mathbf{G}_i) - 0]^2 \right\}, \quad (x_j^i, y_j^i) \in \mathbf{X}_{\text{BC},i}. \end{aligned} \quad (29)$$

4.2.2. A 2D plate problem

In this example, we consider the placement of bolt holes in a 2D plate under varying loading conditions. This is a more complex problem which involves a system of differential equations and a nonlinear mapping is to be learned between multiple parameter functions and multiple response functions. In particular, let us consider a solid mechanics problem for a two-dimensional plate with four holes. The static solution for the plate displacements is governed by the following system of partial differential equations

$$\begin{aligned} \frac{E}{1-\mu^2} \left[\frac{\partial^2 u(x, y)}{\partial x^2} + \frac{(1-\mu)}{2} \frac{\partial^2 u(x, y)}{\partial y^2} + \frac{(1+\mu)}{2} \frac{\partial^2 v(x, y)}{\partial x \partial y} \right] &= 0, \quad (x, y) \in \Omega, \\ \frac{E}{1-\mu^2} \left[\frac{\partial^2 v(x, y)}{\partial y^2} + \frac{(1-\mu)}{2} \frac{\partial^2 v(x, y)}{\partial x^2} + \frac{(1+\mu)}{2} \frac{\partial^2 u(x, y)}{\partial x \partial y} \right] &= 0, \quad (x, y) \in \Omega, \end{aligned} \quad (30)$$

where u and v are the plate displacements in x and y directions, respectively, E is the Young's Modulus, and μ is the Poisson's Ratio.

In this experiment, the plate is considered to be 20 mm $\times$ 20 mm, and the location and size of the bolts are variable. To generate variable geometries, we first consider four circles of radius of 1.5 centered at (5, 5), (5, -5), (-5, 5), and (-5, -5) respectively (shown by dashed lines in Fig. 7). In each circle, we randomly select one point to be the center of a bolt hole. The radius of the holes are uniformly drawn from the range [0.8, 1.5]. The edge of the holes are subjected to the "fixed" boundary condition (zero displacements). The left and right sides of the plate are subject to a prescribed imposed displacements (given as variable functions), while the top and bottom sides are assigned the "free" boundary conditions (zero strains).

The complete boundary conditions are shown in Fig. 7, and are given by

$$\begin{aligned} u(x, y) &= 0, \quad v(x, y) = 0, \quad (x, y) \in \partial\Omega_H, \\ u(x, y) &= u_L(x, y), \quad v(x, y) = v_L(x, y), \quad (x, y) \in \partial\Omega_L, \\ u(x, y) &= u_R(x, y), \quad v(x, y) = v_R(x, y), \quad (x, y) \in \partial\Omega_R, \\ \frac{\partial u(x, y)}{\partial y} &= 0, \quad \frac{1}{2} \left[\frac{\partial u(x, y)}{\partial y} + \frac{\partial v(x, y)}{\partial x} \right] = 0, \quad (x, y) \in \partial\Omega_{\text{TB}}, \end{aligned} \quad (31)$$

where the prescribed displacement functions u_L, v_L, u_R, v_R are the input functions to the operator. Similar to the previous example, these functions are considered to follow a Gaussian Process model. The covariance kernel is assumed to be dependent only on the vertical component of the distance, i.e.,

$$\begin{aligned} u_l, v_l, u_R, v_R &\sim \mathcal{GP}(1, K(y, y')), \\ K(y, y') &= \exp \left[-\frac{(y - y')^2}{2l^2} \right], \quad l = 5. \end{aligned} \quad (32)$$

Similarly to the previous example, we draw $N = 500$ realizations of these functions, and obtain FEM solutions using meshes with different sizes. Fig. 7 shows the scheme of the variable geometry generation and the boundary conditions settings. Some

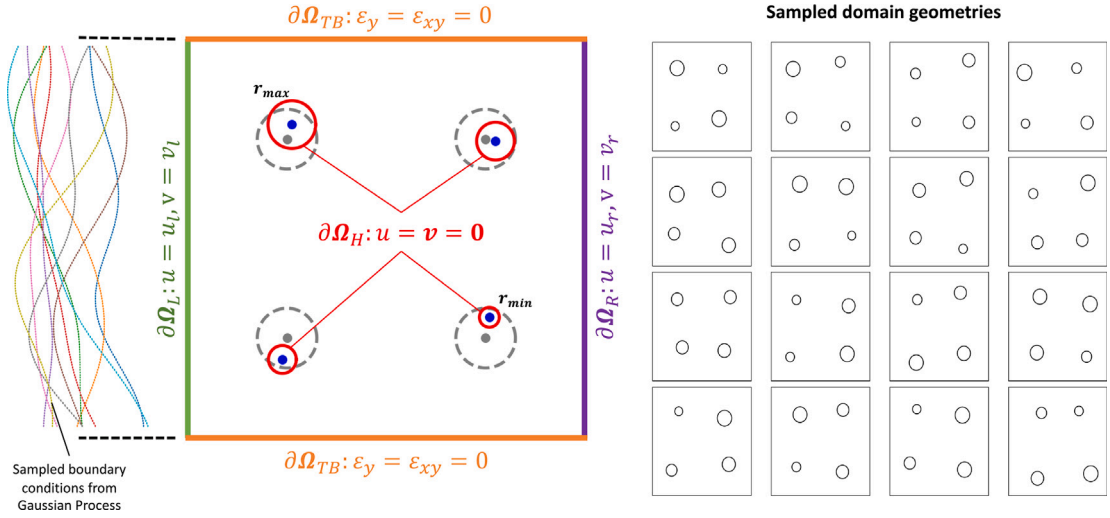


Fig. 7. The boundary conditions of 2D Plate experiment, and 2D plates with largest and smallest bolt hole area are shown. The samples of Gaussian processes as the prescribed displacement on leftmost edges and rightmost edges are also shown. The dots on the curve are considered as the representation of the prescribed displacements.

randomly generated geometries are also shown on the right. The boundary condition information on the left and right sides for the i th realization is collected into the following four vectors

$$\begin{aligned} X_{BC_{TB},i} &= \{(x_j^{1,i}, y_j^{1,i})\}_{j=1}^{M_{1,i}}, \\ X_{BC_{L},i} &= \{(x_j^{2,i}, y_j^{2,i})\}_{j=1}^{M_{2,i}}, \\ X_{BC_{R},i} &= \{(x_j^{3,i}, y_j^{3,i})\}_{j=1}^{M_{3,i}}, \\ X_{BC_{H},i} &= \{(x_j^{4,i}, y_j^{4,i})\}_{j=1}^{M_{4,i}}, \end{aligned} \quad (33)$$

where $X_{BC_{1,i}}$, $X_{BC_{2,i}}$, $X_{BC_{3,i}}$, $X_{BC_{4,i}}$ are the sets of boundary coordinates sampled on $\partial\Omega_1$, $\partial\Omega_2$, $\partial\Omega_3$, $\partial\Omega_4$, respectively. The discretized version of the input functions to the operator (sampled boundary values on the left and right sides) are then represented by the following matrices

$$\begin{aligned} G_{L,i} &= \{X_{BC_{L},i}, u_{L,i}(X_{BC_{L},i}), v_{L,i}(X_{BC_{L},i})\} \\ G_{R,i} &= \{X_{BC_{R},i}, u_{R,i}(X_{BC_{R},i}), v_{R,i}(X_{BC_{R},i})\} \end{aligned} \quad (34)$$

Given this setup, we seek to learn the nonlinear mapping $\mathcal{M}_{\text{plate}}$ that transforms the given boundary displacement functions $u_L(x, y)$, $v_L(x, y)$, $u_R(x, y)$, $v_R(x, y)$ on $\partial\Omega_2$ and $\partial\Omega_3$ to the displacement field $u(x, y)$ and $v(x, y)$ in the entire domain, i.e.,

$$\mathcal{M}_{\text{plate}} : [u_L(x, y), v_L(x, y), u_R(x, y), v_R(x, y)] \rightarrow [u(x, y), v(x, y)]. \quad (35)$$

Based on these boundary condition settings, the training loss function $\mathcal{L}(\theta)$ for training of neural operators $u_\theta(x, y, \mathbf{G}_L, \mathbf{G}_R)$ and $v_\theta(x, y, \mathbf{G}_L, \mathbf{G}_R)$ is formulated as:

$$\mathcal{L}(\theta) = \alpha_0 \mathcal{L}_{\text{PDE}}(\theta) + \alpha_1 \mathcal{L}_{BC_{TB}}(\theta) + \alpha_2 \mathcal{L}_{BC_L}(\theta) + \alpha_3 \mathcal{L}_{BC_R}(\theta) + \alpha_4 \mathcal{L}_{BC_H}(\theta), \quad (36)$$

where $\alpha_0, \alpha_1, \alpha_2, \alpha_3, \alpha_4$ are the trade-off coefficients, set to be $\alpha_0 = 0.00001, \alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = 1$ to ensure similar orders of magnitude; and $\mathcal{L}_{\text{PDE}}(\theta)$ is the PDE residual given by

$$\begin{aligned} \mathcal{L}_{\text{PDE}}(\theta) &= \frac{1}{N} \sum_{i=1}^N \left\{ \frac{E}{1-\mu^2} \left(\frac{\partial^2 u_\theta(x, y, \mathbf{G}_{L,i}, \mathbf{G}_{R,i})}{\partial x^2} + \frac{\partial^2 v_\theta(x, y, \mathbf{G}_{L,i}, \mathbf{G}_{R,i})}{\partial y^2} \right) + \right. \\ &\quad \frac{E}{2(1+\mu)} \left(\frac{\partial^2 u_\theta(x, y, \mathbf{G}_{L,i}, \mathbf{G}_{R,i})}{\partial y^2} + \frac{\partial^2 v_\theta(x, y, \mathbf{G}_{L,i}, \mathbf{G}_{R,i})}{\partial x^2} \right) + \\ &\quad \left. \frac{E}{2(1-\mu)} \left(\frac{\partial^2 v_\theta(x, y, \mathbf{G}_{L,i}, \mathbf{G}_{R,i})}{\partial x \partial y} + \frac{\partial^2 u_\theta(x, y, \mathbf{G}_{L,i}, \mathbf{G}_{R,i})}{\partial x \partial y} \right) \right\}, \quad (x, y) \in \Omega. \end{aligned} \quad (37)$$

Table 2

Accuracy and efficiency comparison between PI-GANO and PI-PointNet for the dataset of only geometry variation.

Dataset	Mean of relative error		Standard deviation of relative error		Training time (min)	
	PI-PointNet	PI-GANO	PI-PointNet	PI-GANO	PI-PointNet	PI-GANO
Darcy flow	11.30%	6.70%	3.52%	2.09%	32	18
2D Plate	25.72%	6.41%	10.13%	2.19%	265	57

The BC loss terms $\mathcal{L}_{\text{BC}_{\text{TB}}}(\theta)$, $\mathcal{L}_{\text{BC}_L}(\theta)$, $\mathcal{L}_{\text{BC}_R}(\theta)$, $\mathcal{L}_{\text{BC}_H}(\theta)$ are given by

$$\begin{aligned}
\mathcal{L}_{\text{BC}_{\text{TB}}}(\theta) &= \frac{1}{N} \sum_{i=1}^N \left\{ \frac{1}{M_{1,i}} \sum_{j=1}^{M_{1,i}} \left[u_{\theta}(x_j^i, y_j^i, \mathbf{G}_{L,i}, \mathbf{G}_{R,i}) - 0 \right]^2 + \left[v_{\theta}(x_j^i, y_j^i, \mathbf{G}_{L,i}, \mathbf{G}_{R,i}) - 0 \right]^2 \right\}, \quad (x_j^i, y_j^i) \in \mathbf{X}_{\text{BC}_{\text{TB},i}}, \\
\mathcal{L}_{\text{BC}_L}(\theta) &= \frac{1}{N} \sum_{i=1}^N \left\{ \frac{1}{M_{2,i}} \sum_{j=1}^{M_{2,i}} \left[u_{\theta}(x_j^i, y_j^i, \mathbf{G}_{L,i}, \mathbf{G}_{R,i}) - u_{L,i}(x_j^i, y_j^i) \right]^2 + \left[v_{\theta}(x_j^i, y_j^i, \mathbf{G}_{L,i}, \mathbf{G}_{R,i}) - v_{L,i}(x_j^i, y_j^i) \right]^2 \right\}, \\
&\quad (x_j^i, y_j^i) \in \mathbf{X}_{\text{BC}_{L,i}}, \\
\mathcal{L}_{\text{BC}_R}(\theta) &= \frac{1}{N} \sum_{i=1}^N \left\{ \frac{1}{M_{3,i}} \sum_{j=1}^{M_{3,i}} \left[u_{\theta}(x_j^i, y_j^i, \mathbf{G}_{L,i}, \mathbf{G}_{R,i}) - u_{R,i}(x_j^i, y_j^i) \right]^2 + \left[v_{\theta}(x_j^i, y_j^i, \mathbf{G}_{L,i}, \mathbf{G}_{R,i}) - v_{R,i}(x_j^i, y_j^i) \right]^2 \right\}, \\
&\quad (x_j^i, y_j^i) \in \mathbf{X}_{\text{BC}_{R,i}}, \\
\mathcal{L}_{\text{BC}_H}(\theta) &= \frac{1}{N} \sum_{i=1}^N \left\{ \frac{1}{M_{4,i}} \sum_{j=1}^{M_{4,i}} \left[\frac{\partial v_{\theta}(x_j^i, y_j^i, \mathbf{G}_{L,i}, \mathbf{G}_{R,i})}{\partial y_j^i} - 0 \right]^2 + \right. \\
&\quad \left. \left[\frac{1}{2} \left(\frac{\partial u_{\theta}(x_j^i, y_j^i, \mathbf{G}_{L,i}, \mathbf{G}_{R,i})}{\partial y_j^i} + \frac{\partial v_{\theta}(x_j^i, y_j^i, \mathbf{G}_{L,i}, \mathbf{G}_{R,i})}{\partial x_j^i} \right) - 0 \right]^2 \right\}, \quad (x_j^i, y_j^i) \in \mathbf{X}_{\text{BC}_{H,i}}.
\end{aligned} \tag{38}$$

4.3. Main results

For both the Darcy flow problem and the 2D Plate problem, we will generate two datasets: one with varying geometry only, and the other one with both varying PDE parameters and varying geometry. The first dataset is used to only investigate geometry generalization and compare the performance of our proposed model with that of PI-PointNet. The second dataset is utilized to evaluate the parameter and geometry generalization abilities of our proposed model with PI-DCON and PI-PointNet*. It should be noted that for the 2D plate dataset, to properly discretize the boundary around the small holes, we employ a higher resolution compared to the Darcy problem. Typically, the meshes in the Darcy flow dataset contain approximately 1000 nodes, whereas the meshes in 2D plate dataset has about 20,000 nodes. We use relative L_2 error [33] as our error measure computed by:

$$L_2 = \frac{\|u_{\text{pred}} - u_{\text{fem}}\|_2}{\|u_{\text{fem}}\|_2}, \tag{39}$$

where u_{pred} refers to a vector of neural network predictions at all the collocation points, and u_{fem} is the corresponding FEM solution vector. We present the average error calculated across the entire test dataset, along with the standard deviation of the relative errors to demonstrate the model stability.

Table 2 shows the results for the case where only geometry generalization is compared with PI-PointNet. As can be seen, our model achieved 39.1% accuracy improvement for the Darcy flow problem and 75.0% accuracy improvement for the 2D Plate problem. Fig. 8, in more details, compares the two models, and show the our method in all the geometry samples offers smaller error compared to PI-PointNet.

Comparing the training times of the two models, we demonstrate that our method surpasses PI-PointNet in terms of efficiency, especially when handling fine meshes, noting the results for the 2D plate problem. In physics-informed training, the computation of PDE residuals is required in each iteration. In our proposed model, we can sample a small subset of the collocation points, in the trunk net, for the approximation calculation of loss and an efficient update of the model parameters. This is because the trunk net, which approximates the solution at a given point, is chosen to be separate from the geometry encoder. In contrast, PI-PointNet can only handle the entire point cloud representing the entire domain. Therefore, stochastic gradient descent algorithm with a small minibatch size cannot work for this model architecture. As a result, we observe a significantly reduced training time for our model when dealing with fine meshes. To visually compare our method with PI-PointNet, the best and worst-case predictions are presented in Table 3.

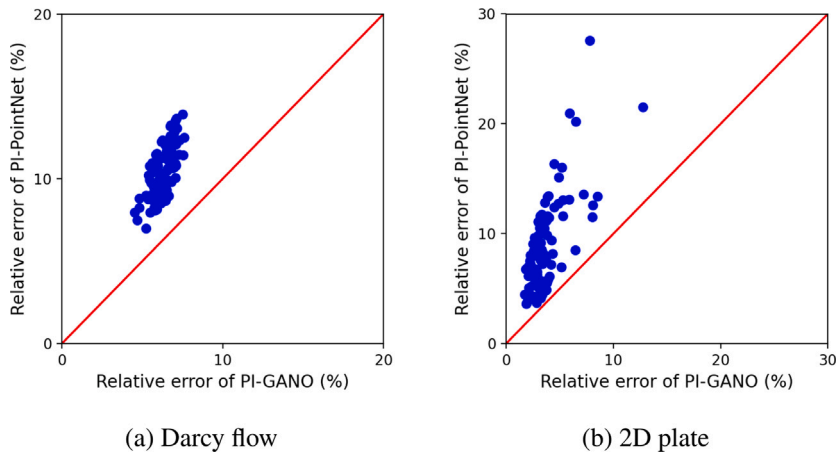


Fig. 8. Comparison between PI-GANO and PI-PointNet on the Darcy flow and 2D plate problems. It can be seen that in all the cases PI-GANO offers the smaller errors.

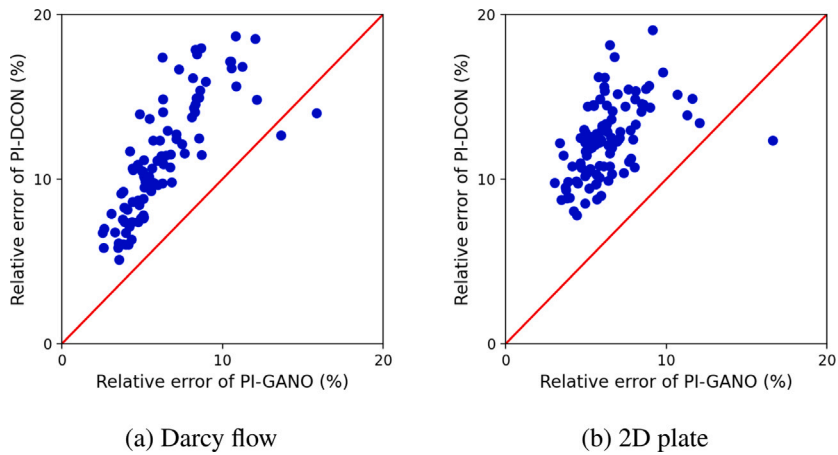


Fig. 9. Comparison between PI-GANO and PI-DCON on the Darcy flow and 2D plate problems. It can be seen that in all the cases PI-GANO offers the smaller errors.

For cases where both PDE parameters and geometries vary, we compare our method with PI-DCON and PI-PointNet* in [Table 4](#). It can be seen that the PI-PointNet* struggles to achieve satisfactory accuracy, underscoring the challenge of developing a model capable of generalizing to varying PDE parameters and geometries. Our proposed model demonstrated highest accuracy in addressing both the Darcy flow problem and the 2D Plate problem. Compared to PI-DCON, our model achieved a 60% accuracy improvement for the Darcy flow problem and a 50% accuracy improvement for the 2D Plate problem, illustrating the effectiveness of the geometry encoder in our model. [Fig. 9](#) shows more detailed comparison between the errors calculated for each testing samples, and underlines the superiority of PI-GANO in all the test cases.

To better illustrate the performance of our model, [Table 5](#) shows the predictions from PI-DCON and PI-GANO and their errors compared to the FEM prediction, for various best and worst-case scenarios. Overall, our model shows higher accuracy maintaining acceptable performance even in its worst cases. Furthermore, the relatively small performance gaps between the worst and best cases underscore the robustness of our model.

4.4. Ablation study: Different geometry embeddings

In previous experiments, we used the collocation points on the boundary to infer important features of domain geometry. These collocation points can be collected from different parts of the domain. In particular, in this section, we study four different ways to do so: (1) collecting collocation points on segments of the boundary that vary, (2) collecting collocation points on all boundary segments, (3) collecting collocation points in the entire domain. As an additional case, we also consider using directly the parametric representation of the geometry, which in our examples are available.

The parametric geometry representation of the Darcy flow problem consists of the coordinates of the five points that form the polygon, i.e. a total of 10 coordinates. Similarly, for the 2D plate problem, the parametric geometry representation contains

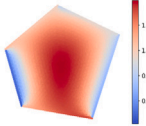
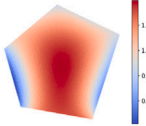
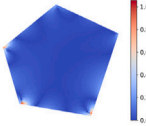
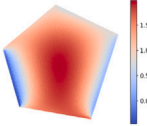
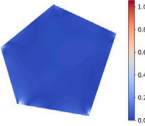
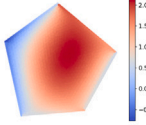
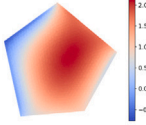
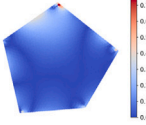
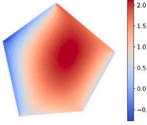
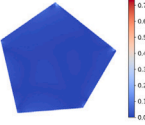
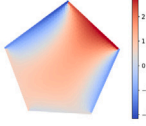
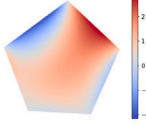
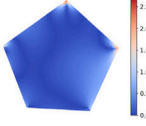
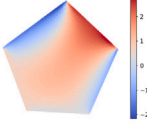
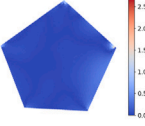
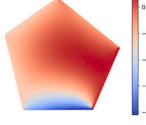
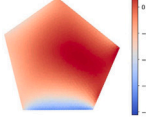
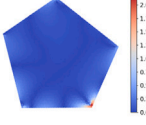
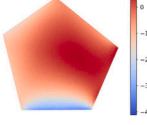
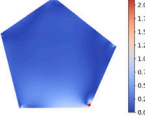
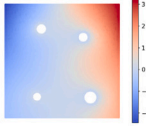
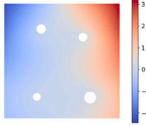
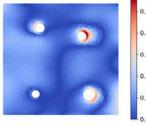
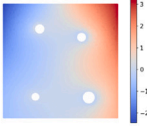
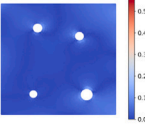
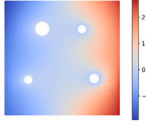
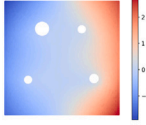
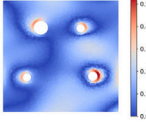
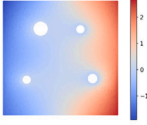
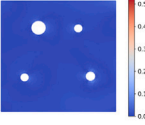
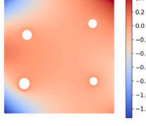
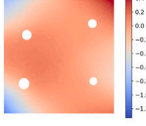
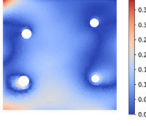
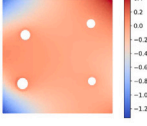
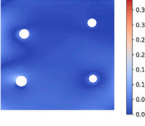
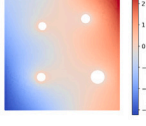
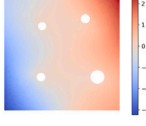
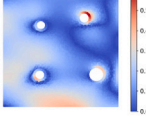
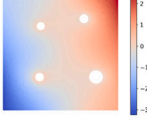
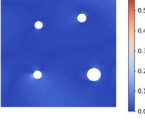
Table 3
Comparison between the performance of PI-GANO and PI-PointNet. Out of all realizations of boundary conditions, the ones that causes best and worst performances of each model are shown.

		Ground Truth	PI-PointNet		PI-GANO	
			Prediction	Absolute Error	Prediction	Absolute Error
Darcy flow	PI-PointNet Best case					
	PI-GANO Best case					
	PI-PointNet Worst case					
	PI-GANO Worst case					
2D Plate	PI-PointNet Best case					
	PI-GANO Best case					
	PI-PointNet Worst case					
	PI-GANO Worst case					

Table 4
Accuracy comparison between PI-GANO and PI-DCON.

Dataset	Mean of relative error			Standard deviation of relative error		
	PI-PointNet*	PI-DCON	PI-GANO	PI-PointNet*	PI-DCON	PI-GANO
Darcy flow	64.3%	13.50%	8.40%	24.7%	5.88%	2.53%
2D Plate	74.3%	14.06%	7.12%	27.8%	4.81%	2.82%

Table 5
Comparison between the performance of PI-GANO and PI-DCON. Out of all realizations of boundary conditions, the ones that causes best and worst performances of each model are shown.

		Ground Truth	PI-DCON		PI-GANO	
			Prediction	Absolute Error	Prediction	Absolute Error
Darcy flow	PI-DCON Best case					
	PI-GANO Best case					
	PI-DCON Worst case					
	PI-GANO Worst case					
2D Plate	PI-DCON Best case					
	PI-GANO Best case					
	PI-DCON Worst case					
	PI-GANO Worst case					

the locations and sizes of the four holes, i.e. a total of 12 parameters. For cases where the parametric representation is used, the geometry encoder will be simply a multi-layer perceptron.

As shown in Table 6, the highest model accuracy is achieved when using collocation points from varying boundaries. Although using all collocation points provides the most comprehensive information about the domain geometry, this approach results in the poorest model performance. This can be due to the fact that many interior point locations are common across different geometries, rendering them redundant in capturing distinct geometric features leading to deterioration in effective geometry encoding. Furthermore, when comparing model performance using points from varying boundaries and parametric representations, the use of varying boundary points leads to 1.7% accuracy improvement for the Darcy flow problem and 20% improvement for the 2D Plate problem, indicating that parametric representations is not necessarily always the most effective approach.

Table 6

Mean of relative error of different geometry embedding usage.

Geometry input data	Interior and boundary points	Boundary points (fixed and variable)	Parameters of parametric representation	Boundary points (variable only)
Darcy flow	11.1%	/	8.56%	8.40%
2D plate	21.17%	14.81%	9.02%	7.21%

Table 7

Mean of relative error of different geometry embedding methods.

Problem	Operations		
	Multiplication	Addition	Concatenation
Darcy flow	11.13%	10.90%	8.40%
2D plate	15.17%	13.81%	7.21%

Table 8

Mean of relative error of different pooling layers.

Problem	Operations		
	Min-pooling	Max-pooling	Avg-pooling
Darcy flow	8.49%	8.44%	8.40%
2D plate	7.38%	7.27%	7.21%

Table 9

Accuracy comparison between PI-GANO and PI-DCON for the dataset of high geometry variation.

Model	Mean of relative error	Standard deviation of relative error
PI-DCON	23.72%	10.91%
PI-GANO	7.29%	3.22%

In our proposed architecture, we used concatenation to combine the geometry embedding with the local coordinate embedding. Here, we study the performance of the model when these two embeddings are integrated using other forms. In particular, we consider the following three cases

$$\text{Multiplication : } \mathbf{H}_j^i = \mathbf{h}_j^i \odot \mathbf{G}^i, \quad (40)$$

$$\text{Addition : } \mathbf{H}_j^i = \mathbf{h}_j^i + \mathbf{G}^i, \quad (41)$$

$$\text{Concatenation : } \mathbf{H}_j^i = [\mathbf{h}_j^i \parallel \mathbf{G}^i], \quad (42)$$

where $\mathbf{G}^i$ is the global feature of the i th geometry, $\mathbf{h}_j^i$ is the local hidden embedding of the j th coordinate in the i th geometry, and $\mathbf{H}_j^i$ is the global hidden embedding of the j th coordinate in the i th geometry. To generate the results, we only use collocation points on the varying boundary, which was shown earlier to be the best approach. As can be seen in Table 7, concatenation offers the highest accuracy.

We also investigate the impact of various types of pooling on the model's performance. By incorporating boundary points as geometry information and utilizing concatenation operations, we assess the prediction accuracy for different pooling types, as shown in Table 8. Our findings reveal that employing an average pooling layer results in the slightly enhanced performance, compared to other pooling types. We believe this improvement is because that Average pooling maintains more consistent gradients during back-propagation compared to max-pooling and min-pooling, which can lead to smoother and more stable learning.

4.5. Generalization for large geometry variation

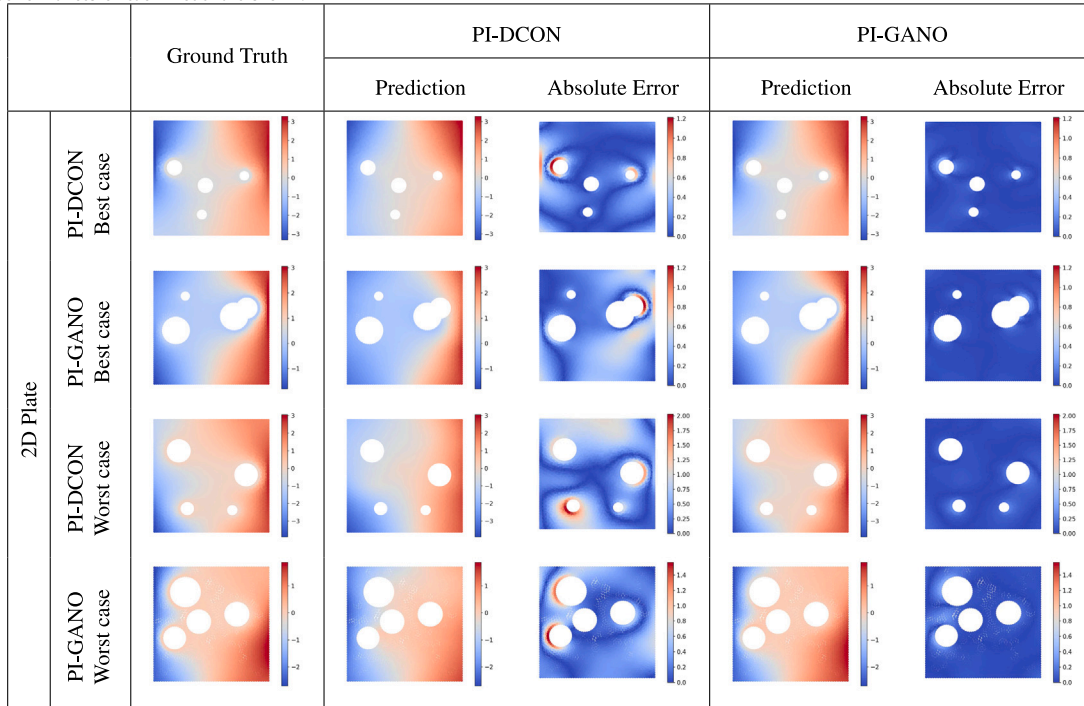
In this section, we further explore the geometry generalization using a dataset with significantly higher geometry variation. This dataset is created under the same settings as those described in Section 4.2.2, with the exception that the holes can be of any radius and they can be placed anywhere on the plate, as long as they do not protrude beyond the outer edges. Overlapping holes are also allowed in this data generation, merely to create a more challenging generalization test. To ensure comprehensive coverage of the highly variable geometric distribution, we generate 5000 different geometry samples. The size ratio of the training, validation and testing datasets are 70%, 10%, and 20%, respectively.

As indicated in Table 9, our proposed model achieves approximately a 70% accuracy improvement on this challenging dataset, underscoring its robustness in handling highly varying geometries. By visually comparing the model's performances, Table 10 shows that PI-GANO can offer more accurate predictions for various best and worst-case scenarios.

To evaluate the model's capability to handle geometry test samples that are differently distributed compared to the training data, we performed several experiments using various training-testing dataset pairs. These includes transfer learning cases where the model was trained on low-variation geometry samples and tested on high-variation geometry samples, and vice versa, as well

Table 10

Comparison between the performance of PI-DCON and PI-DeepONet. Out of all realizations of boundary conditions, the ones that causes best and worst performances of each model are shown.

**Table 11**

Accuracy comparison of PI-GANO between using low-variation training dataset and high-variation training dataset.

	Training set	Test set	Mean of relative error	Standard deviation of relative error
Case 1	500 low-var	100 high-var	59.72%	32.91%
Case 2	500 high-var	100 low-var	47.12%	22.91%
Case 3	500 low & high	100 low-var	23.8%	6.4%
	500 low & high	100 high-var	33.8%	12.5%

as another ensemble case where the model was trained on a set which consisted of both low- and high-variation geometry samples. As shown in Table 11, the model fails to provide accurate predictions when the training and testing datasets come from two distinct distributions. This is while training the model with both datasets leads to performance improvements. However, the performance is still inferior compared to the case where both training and testing data are from the same distribution, indicating that the model has limitations in handling out of distribution geometry variations.

We also visualize the differences between the two geometry distributions that we studied, by analyzing their geometry embeddings. In Fig. 10, we show the distance of the hidden embeddings of both low-variation and high-variation geometry samples, measured from the center of the cluster of low-variation geometry samples. Three different training-test cases shown in Table 11 were used. In Cases 1 and 2, it is evident that the average distance of the high-variation samples is nearly four times as that of the low-variation samples, emphasizing that the two geometry distributions are significantly different. Even when the model is trained on both low-variation and high-variation samples (Case 3), we can still see the significant difference between the learned embeddings of low-variation and high-variation samples.

4.6. Comparison with data-driven neural operators

In this section, we evaluate the efficiency and accuracy of our proposed physics-informed model compared to the data-driven (supervised) versions of DCON and GANO architectures. In terms of inference time, solving a Darcy problem using FEM takes approximately 1 s, while a 2D plate stress problem requires 2.1 s. In contrast, predicting the PDE solutions with the neural operator takes only 0.0021 s and 0.0038 s, respectively, demonstrating an efficiency improvement of three orders of magnitude. However, beyond inference efficiency, our primary focus is on the computational cost associated with training the models and how a physics-informed can be advantageous.

We use the same dataset of 500 samples, as in our previous experiments, with 70% allocated for training, 10% for validation, and 20% for testing. The total time for FEM data generation is estimated based on 350 FEM computations. We report the accuracy

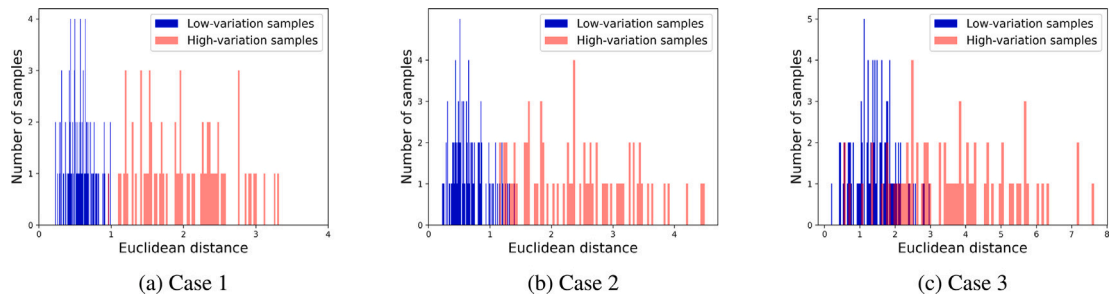


Fig. 10. The histograms of Euclidean distance between the embedding vector of various geometry samples. The distance is measured with respect to the center of cluster of embedding vectors corresponding to the low-variation geometry samples. The three cases are defined in Table 11.

Table 12

Accuracy comparison between data-driven DCON and data-driven GANO.

Dataset	Mean of relative error		Standard deviation of relative error	
	Data-driven DCON	Data-driven GANO	Data-driven DCON	Data-driven GANO
Darcy flow	8.21%	8.00%	3.27%	2.93%
2D plate	13.5%	9.01%	4.56%	3.17%

of the two data-driven models in Table 12, where the proposed GANO architecture offers superior accuracy over DCON. We also present the best model performance achieved versus the total time of building the model in Fig. 11. For the data-driven models, the total time includes the data generation and training times. It can be seen that for the Darcy flow problem, PI-GANO reaches a convergence accuracy comparable to that of data-driven GANO and DCON, while requiring less time to achieve this accuracy level. This underscores the efficiency benefits of using physics-informed training to develop neural operators. Additionally, data-driven GANO and data-driven DCON exhibit similar prediction accuracy levels. We attribute this to the fact that the variable PDE parameter, which are the boundary conditions, in the Darcy flow problem, are imposed on the entire exterior boundary of the domain. This means that the PDE parameter input also inherently contains geometry information about the entire geometry. Therefore, the addition of a separate geometry encoder does not significantly enhance the model performance.

On the other hand, for the 2D plate problem, data-driven GANO achieves over a 28% accuracy improvement compared to data-driven DCON. This shows it is important to include a separate geometry encoder, because the variable parameters in this problem are only imposed on a subset of boundary segments, which do not fully characterize the entire domain. Furthermore, it can be seen in this figure that our proposed PI-GANO outperforms the data-driven version of GANO, highlighting the potential of physics-informed training to offer efficiency and also to enhance prediction accuracy when the same model architecture is used.

An interesting observation is that PI-GANO outperforms the data-driven GANO in the 2D plate stress problem. Typically, we would expect data-driven training to yield better performance than physics-informed training when using the same model architecture, as it is a more straightforward and easily implementable approach. However, as shown in works related to PI-DeepONet [17,34], physics-informed models can sometimes surpass data-driven models when the model architecture and training algorithm are appropriately combined. Additionally, this comparison is constrained by equal training times, suggesting that the data-driven model might outperform the physics-informed model with extended training time.

We also observe that, in the 2D plate problem, the best model prediction accuracy of the physics-informed models does not always decrease with more training epochs. Instead, it may remain unchanged during parts of the training process. This phenomenon can be attributed to the small size of the collocation point sampling used for PDE residual computation to handle meshes of high resolutions. Using a small set of points for PDE residual computation can cause the model to overly focus on the accuracy of a small region in the domain, at the expense of overall approximation accuracy across the entire domain. This issue highlights the need for more advanced physics-informed training algorithms to enhance the training of neural operators.

4.7. Hyper-parameter studies

In this section, we analyze the impact of various hyperparameters on the model's performance in terms of geometry generalization. Specifically, we focus on the number of hidden layers in the geometry encoder and the size of the training dataset. As illustrated in Fig. 12, model performance improves with increasing complexity of the geometry encoder and a larger training dataset. This result is intuitive, as a more complex geometry encoder allows for better feature extraction from the domain geometry, and a larger dataset provides the model with more opportunities to learn and generalize. However, we also observe a significant drop in performance when the training dataset is insufficient. For instance, when the number of training samples decreases from 150 to 50, the model's average prediction error rises from approximately 17% to 40%.

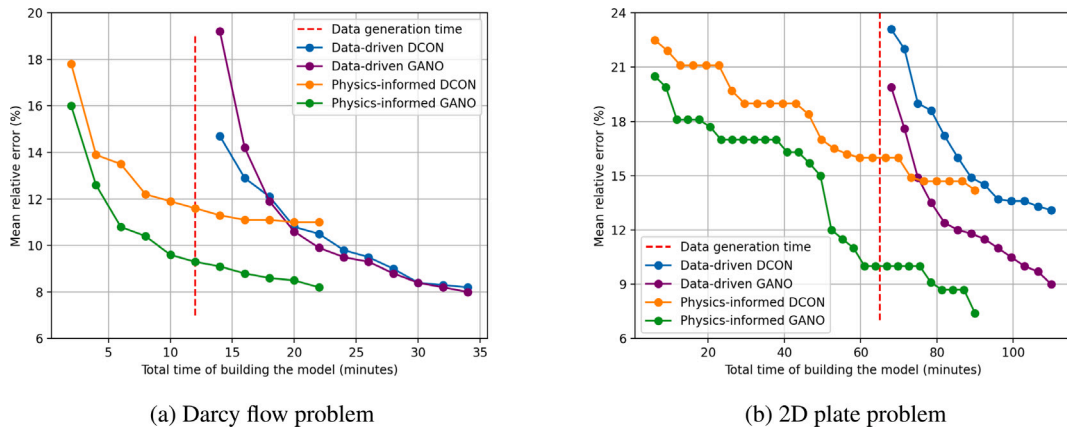


Fig. 11. Comparison between the total time needed to train different neural operators. For data-driven method, this time also includes the time needed to generate the training (FE simulation) data.

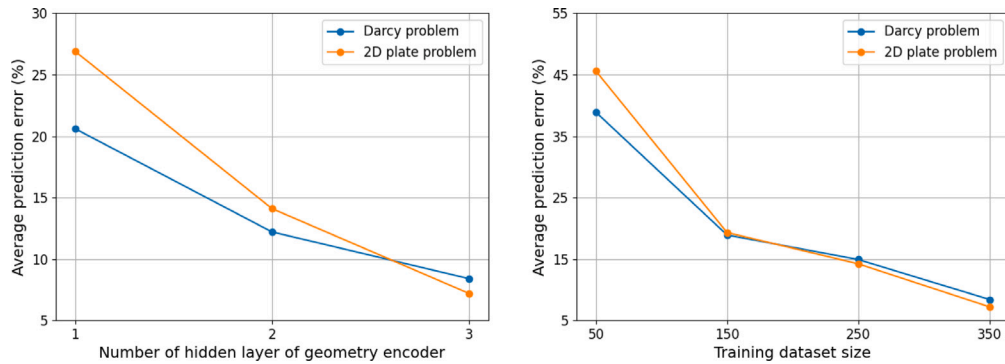


Fig. 12. Hyper-parameter studies of PI-GANO architecture.

5. Conclusion

In this study, we introduced PI-GANO, a physics-informed neural operator inspired by PI-PointNet and PI-DCON, marking the first attempt to construct a neural operator capable of generalizing to both variable PDE parameters and variable domain geometries without the need for FEM training data or a regular meshing of domains. Our results demonstrate that PI-GANO can effectively predict PDE solutions for variable boundary conditions and domain geometries. Moreover, comparing the proposed PI-GANO with the data-driven version of our proposed architecture, we showed the potential of physics-informed training to offer efficiency and accuracy benefits in developing neural operators.

Despite these advancements, our approach has its limitations. Primarily, our evaluation focuses solely on the model's capability to compute steady-state solutions. Further research is needed to adapt PI-GANO to handle dynamic responses, thereby enabling it to address time-dependent PDEs. Additionally, we observed instability in physics-informed training, which leads to requiring more training epochs to identify optimal model parameters. To address this, we seek to adopt a more intelligent collocation point sampling strategy, importance sampling [35], for physics-informed training. Moreover, this approach can be investigated in more challenging problems such as those with cracks or highly nonlinear responses.

Furthermore, the current model is restricted to solving PDE problems within a specific underlying distribution of geometries and PDE parameters, as is common in many design problems with limited variation. However, to tackle scenarios where domain geometries and problem settings display greater variability or multiple varying PDE parameters, future work will focus on developing a more robust foundation model capable of generalizing across diverse distributions of design parameters. This will require designing a more advanced architecture with increased complexity and capacity, while also supporting physics-informed training. For example, using multiple PDE parameter encoders to capture different PDE parameter information will be one possible improvement. To address more complex problems, we also advocate for the use of advanced physics-informed training algorithms to enhance the training of existing complex data-driven neural operators.

CRediT authorship contribution statement

Weiheng Zhong: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Hadi Meidani:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Methodology, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] Gouri Dhatt, Emmanuel Lefrançois, Gilbert Touzot, Finite Element Method, John Wiley & Sons, 2012.
- [2] Waad Almasri, Dimitri Bettebghor, Faouzi Adjed, Florence Danglede, Fakhreddine Ababsa, Geometrically-driven generation of mechanical designs through deep convolutional GANs, *Eng. Optim.* 56 (1) (2024) 18–35.
- [3] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, George Em Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, *Nat. Mach. Intell.* 3 (3) (2021) 218–229.
- [4] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, Anima Anandkumar, Fourier neural operator for parametric partial differential equations, 2020, arXiv preprint [arXiv:2010.08895](https://arxiv.org/abs/2010.08895).
- [5] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, Anima Anandkumar, Neural operator: Learning maps between function spaces, 2021, arXiv preprint [arXiv:2108.08481](https://arxiv.org/abs/2108.08481).
- [6] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, Anima Anandkumar, Neural operator: Graph kernel network for partial differential equations, 2020, arXiv preprint [arXiv:2003.03485](https://arxiv.org/abs/2003.03485).
- [7] Zongyi Li, Daniel Zhengyu Huang, Burigede Liu, Anima Anandkumar, Fourier neural operator with learned deformations for pdes on general geometries, *J. Mach. Learn. Res.* 24 (388) (2023) 1–26.
- [8] Tapas Tripura, Souvik Chakraborty, Wavelet neural operator: a neural operator for parametric partial differential equations, 2022, arXiv preprint [arXiv:2205.02191](https://arxiv.org/abs/2205.02191).
- [9] Chu Wang, Jinhong Wu, Yanzhi Wang, Zhijian Zha, Qi Zhou, MIPIN: A Multi Physics-Informed PointNet for solving parametric acoustic-structure systems, 2024, arXiv preprint [arXiv:2403.01132](https://arxiv.org/abs/2403.01132).
- [10] Zongyi Li, Nikola Kovachki, Chris Choy, Boyi Li, Jean Kossaifi, Shourya Otta, Mohammad Amin Nabian, Maximilian Stadler, Christian Hundt, Kamyar Azizzadenesheli, et al., Geometry-informed neural operator for large-scale 3d pdes, *Adv. Neural Inf. Process. Syst.* 36 (2024).
- [11] Zhiwei Zhao, Changqing Liu, Yingguang Li, Zhibin Chen, Xu Liu, Diffeomorphism Neural Operator for various domains and parameters of partial differential equations, 2024, arXiv preprint [arXiv:2402.12475](https://arxiv.org/abs/2402.12475).
- [12] Zhongkai Hao, Zhengyi Wang, Hang Su, Chengyang Ying, Yinpeng Dong, Songming Liu, Ze Cheng, Jian Song, Jun Zhu, Gnot: A general neural operator transformer for operator learning, in: *International Conference on Machine Learning*, PMLR, 2023, pp. 12556–12569.
- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [14] Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, Mingsheng Long, Transolver: A fast transformer solver for PDEs on general geometries, 2024, arXiv preprint [arXiv:2402.02366](https://arxiv.org/abs/2402.02366).
- [15] AmirPouya Hemmasian, Amir Barati Farimani, Multi-scale time-stepping of Partial Differential Equations with transformers, *Comput. Methods Appl. Mech. Engrg.* 426 (2024) 116983.
- [16] Seungjun Lee, Taeil Oh, Inducing point operator transformer: A flexible and scalable architecture for solving PDEs, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38, 2024, pp. 153–161.
- [17] Sifan Wang, Hanwen Wang, Paris Perdikaris, Learning the solution operator of parametric partial differential equations with physics-informed DeepONets, *Sci. Adv.* 7 (40) (2021) eabi8605.
- [18] Maziar Raissi, Paris Perdikaris, George E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [19] Bart Van Merriënboer, Olivier Breuleux, Arnaud Bergeron, Pascal Lamblin, Automatic differentiation in ML: Where we are and where we should be going, *Adv. Neural Inf. Process. Syst.* 31 (2018).
- [20] Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, Anima Anandkumar, Physics-informed neural operator for learning partial differential equations, 2021, arXiv preprint [arXiv:2111.03794](https://arxiv.org/abs/2111.03794).
- [21] N. Navaneeth, Tapas Tripura, Souvik Chakraborty, Physics informed WNO, *Comput. Methods Appl. Mech. Engrg.* 418 (2024) 116546.
- [22] Weiheng Zhong, Hadi Meidani, Physics-informed mesh-independent deep compositional operator network, 2024.
- [23] Ali Kashefi, Tapan Mukerji, Physics-informed PointNet: A deep learning solver for steady-state incompressible flows and thermal fields on multiple sets of irregular geometries, *J. Comput. Phys.* 468 (2022) 111510.
- [24] Hossein Gholamalinezhad, Hossein Khosravi, Pooling methods in deep neural networks, a review, 2020, arXiv preprint [arXiv:2009.07485](https://arxiv.org/abs/2009.07485).
- [25] Jean-Gabriel Attali, Gilles Pagès, Approximations of functions by a multilayer perceptron: a new approach, *Neural Netw.* 10 (6) (1997) 1069–1081.
- [26] Mian Mian Lau, King Hann Lim, Review of adaptive activation function in deep neural network, in: *2018 IEEE-EMBS Conference on Biomedical Engineering and Sciences*, IECBES, IEEE, 2018, pp. 686–690.
- [27] Diederik P. Kingma, Jimmy Ba, Adam: A method for stochastic optimization, 2014.
- [28] James Bergstra, Yoshua Bengio, Random search for hyper-parameter optimization, *J. Mach. Learn. Res.* 13 (2) (2012).
- [29] Christopher Z. Mooney, Monte Carlo Simulation, Sage, 1997.
- [30] Emanuel Parzen, Stochastic Processes, SIAM, 1999.
- [31] Starting Matlab, Matlab, The MathWorks, Natick, MA, 2012.

- [32] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, George Em Karniadakis, A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data, *Comput. Methods Appl. Mech. Engrg.* 393 (2022) 114778.
- [33] Pooneh Heydari, Mohammad S. Bagheri, Error analysis: Sources of L2 learners' errors, *Theory Pract. Lang. Stud.* 2 (8) (2012) 1583–1589.
- [34] Sifan Wang, Hanwen Wang, Paris Perdikaris, Improved architectures and training algorithms for deep operator networks, *J. Sci. Comput.* 92 (2) (2022) 35.
- [35] Mohammad Amin Nabian, Rini Jasmine Gladstone, Hadi Meidani, Efficient training of physics-informed neural networks via importance sampling, *Comput.-Aided Civ. Infrastruct. Eng.* 36 (8) (2021) 962–977.